This document describes the process of porting this ML-KEM implementation. It is portable so that it can be ported to any architecture that meets the following conditions:

1. The target platform must provide a C11-compatible compilation environment and the basic types and memory operations required by the implementation.
2. Platform-specific services such as allocation, entropy, secure zeroization, atomic operations, and error mapping are provided through the porting interface described below.

Selecting the target platform

In order for the implementation to work, you must first attach the port header and define a preprocessor macro that, when selected, will attach the desired port header during compilation. This should be done in ml_kem_core_header.h in this part of the code:

```
    // HEADERS
#if defined(USERSPACE)

#include <ml_kem_userspace_port.h>

#elif defined(LINUX_KERNEL)

#include "ml_kem_linux_kernel_port.h"

#elif defined(FREERTOS)

#include "ml_kem_freertos_port.h"

#else
#error "No ML-KEM port selected."
#endif // HEADERS
```

Writing a port header

In the header that will be connected to the #elif directive, you should specify:

1. Required headers from the standard C library, or equivalents for your platform
2. Through typedef, redefine the type for ml_kem_atomic_t. This is required for atomic operations in the implementation against the pool
3. If your platform does not support the types u8, u16, u32, u64 and s16 and s32 for signed and unsigned types, you should override the standard C language types or your environment types as u8, u16, u32, u64 and s16 and s32 via typedef.

Writing wrappers for defined prototypes in ml_kem_core_header.h platform-dependent functions in a separate file

You also need to write a port file that describes and implements the platform-specific prototypes defined in ml_kem_core_header.h:

1. Functions for allocation and clearing memory
   void *ml_kem_alloc(size_t size_alloc)
   void ml_kem_free(void *ptr)
2. A function for getting random data. It is recommended to have reliable entropy. If you don't want to use entropy in your code and override it, you can simply return NULL and pass the entropy function itself via parameters in the API.
   int ml_kem_entropy(void *buf, size_t len)
3. Reliable secret zeroing function. The standard zeroing function via volatile is offered here, however, if you have your own zeroing function in your environment, then due to the specifics of compiler optimizations, it is recommended to use it and override it in the wrapper **void ml_kem_memzero(void *ptr, size_t len)**

4. Functions for atomic operations. You need to define and describe three atomic operation functions. This is necessary for working with slots and ensuring that races and deadlocks are avoided. The functions are as follows:
   bool ml_kem_try_acquire_slot(ml_kem_atomic_t *slot) - Gaining access to a slot
   void ml_kem_release_slot(ml_kem_atomic_t *slot) - Atomic slot release
   void ml_kem_atomic_init_slot(ml_kem_atomic_t *slot) - Atomic initialization of a variable for a slot
5. Error handling. Internally, the implementation has its own error handling. This is done to allow you to write your own error handling for the API when porting. The reason is that there are certain environments with specific error handling, for which a simple return of NULL or POSIX code may not be enough. The wrapper functions for error handling are the following:
   struct ml_kem_pool_decaps_ctx *err_create_keys(const int err) - Takes an error code in the key generation API function and returns a pointer of the corresponding type.
   u8 *err_encaps(const int err) - Takes an error code in the function API for encapsulation and returns a pointer of the corresponding type.
   int err_decaps(const int err) - Accepts the error code in the function API for decapsulation and returns the code defined in the function body
   int err_get_pk(const int err) - Accepts the error code in the API function for obtaining the public key and returns the code defined in the function body
   The other two functions in the API have no prototypes to override because they are void. Nothing is expected to return because they deal with cleaning up resources and reclaiming memory and are in fact destructors.
   You can also change or add your own error codes in the implementation, if it is more convenient for your platform and better

in the wrapper functions used in the API. Changing the internal structure of error handling in the implementation is not recommended.

Assembly and compilation

Here everything depends on your platform and compiler and compilation tool. But it is advisable for you to write the two files mentioned above (header and port code) and together with all the files from the portable/src/ directory (except README, this is not mandatory, I think the compiler will forgive you for this ;) to assemble and compile the project. If you do not want to write platform-dependent files in the way described above for your own reasons or the specifics of the environment, then you can do it in the way you consider necessary, the main platform-dependent code and other specific points are defined and described above. I hope this short instruction helped you